
FIELD NOTES / APPLICATION SECURITY · PART 2
$ recon --target newprogram.com
[✓] scope verified
[✓] subdomains enumerated: 142
[✓] live hosts probed: 58
[✓] tech stack fingerprinted
[✓] attack surface mapped
[ ] manual testing — prioritized
0 hours wasted on the wrong target
THE RECON WORKFLOW I BUILT AFTER 10 REJECTIONS
— six phases, zero wasted hours
PART 2 OF THE BUG BOUNTY FIELD NOTES SERIES

June 22, 2026

The Recon Workflow I Built After 10 Rejections

Six phases, zero wasted hours — the process I wish I had on day one.

By Ismail Tasdelen

5 min read

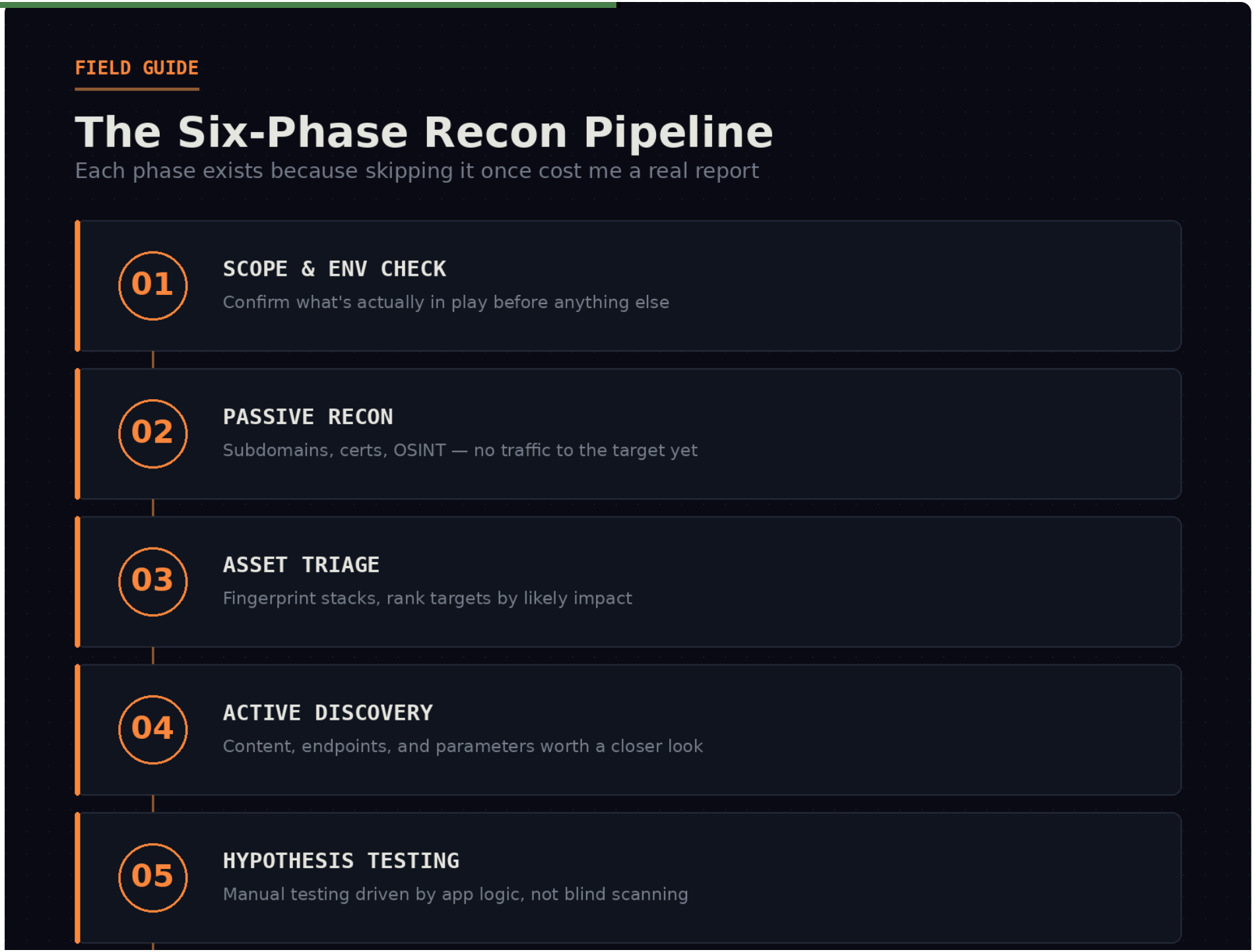
Download article

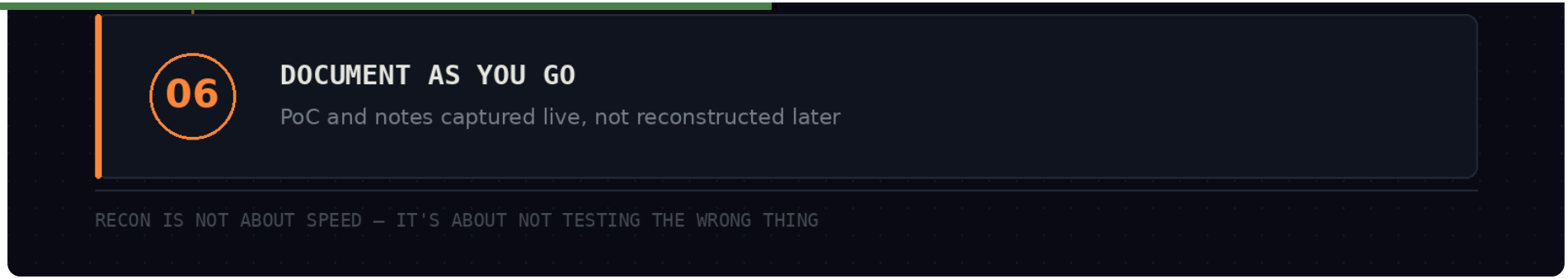
Contents 8

In my last post, I walked through [10 bug bounty reports that got rejected] — duplicates, scope misses, a self-inflicted staging-vs-production mix-up that still makes me wince. A few people asked the same question in the comments: *"Okay, but what do you actually do differently now?"*

Fair question. So here's the answer: I stopped treating recon as a warm-up and started treating it as the actual work. Every phase below exists because skipping it, even once, cost me a real report.

This isn't a tool dump. It's the order I do things in, and why that order matters more than any individual tool.


FIELD GUIDE
The Six-Phase Recon Pipeline
Each phase exists because skipping it once cost me a real report
01
SCOPE & ENV CHECK
Confirm what's actually in play before anything else
02
PASSIVE RECON
Subdomains, certs, OSINT — no traffic to the target yet
03
ASSET TRIAGE
Fingerprint stacks, rank targets by likely impact
04
ACTIVE DISCOVERY
Content, endpoints, and parameters worth a closer look
05
HYPOTHESIS TESTING
Manual testing driven by app logic, not blind scanning



Phase 0: The Rule That Changed Everything

Before any phase, one rule now sits above all of them: **I don't touch a target until I can state, in one sentence, what's in scope and why I believe that.**

This sounds obvious. It wasn't obvious to me [X months ago], when a confident-looking subdomain cost me an afternoon and a rejection. Now it's a hard gate — not a step I rush through to get to the "real" testing.

Phase 1: Scope & Environment Verification

[Describe your usual ritual here — how long you spend, what you check first.]

This phase produces zero bugs and feels unproductive. It's also the one most directly responsible for two of my rejections — an out-of-scope asset and a staging environment mistaken for production. So now, before anything else, I confirm:

- **The scope document itself**, read in full, not skimmed — programs update scope more often than researchers check it.
- **Whether the asset is actually live and customer-facing**, by checking response headers, certificate issuer and validity dates, and any `robots.txt` or `security.txt` hints.
- **Ownership ambiguity** — if a subdomain *could* belong to a third-party vendor or an acquisition, I ask the program before investing real hours, instead of assuming.

Tools that help here: `httpx` for fast header/status checks, `whois` / `crt.sh` for ownership and certificate history, and a five-minute read of the program's own changelog or recent disclosures if one is public.

Phase 2: Passive Reconnaissance

This is the only phase where I generate zero traffic to the target itself — everything comes from public sources.

- **Subdomain enumeration** via certificate transparency logs and public DNS datasets — tools like `subfinder`, `amass` (passive mode), and `assetfinder` cover most of this.
- **OSINT on the organization** — GitHub repos and gists, exposed API documentation, job postings that reveal tech stack, Shodan/Censys for exposed infrastructure.
- **Historical data** — Wayback Machine snapshots often reveal old endpoints, deprecated API versions, or parameters that current crawling won't surface.

[Mention your personal habit here — e.g., how many subdomains a typical target turns up, or a passive-recon trick you rely on.]

The output of this phase is a list, not a finding. That's the point — I'm mapping the territory before I set foot in it.

Phase 3: Asset Triage & Fingerprinting

A list of 150 subdomains is not a plan. This phase turns the list into a priority order.

- **Live-host probing** — `httpx` again, this time to filter the list down to what's actually responding.
- **Technology fingerprinting** — identifying frameworks, CMS platforms, and known libraries in use, since certain stacks correlate strongly with certain bug classes.

- **Screenshotting at scale** — tools like `gowitness` turn a long list of URLs into something I can visually scan in minutes, which surfaces admin panels, staging banners, and forgotten internal tools far faster than reading raw output.

I rank assets by a rough mix of **exposure** (how customer-facing it is), **freshness** (newer code tends to have newer mistakes), and **complexity** (an asset doing real business logic beats a static marketing page every time).

[Add your own ranking heuristic if it differs — this is where personal judgment matters most.]

Phase 4: Active Discovery

Only now do I start sending real, deliberate traffic to the target.

- **Content and endpoint discovery** — `ffuf` or `feroxbuster` against the highest-priority assets, using wordlists tuned to the detected tech stack rather than a generic list.
- **Parameter mining** — tools like `arjun` or manual review of JS bundles to find parameters that aren't visible in the rendered page.
- **API surface mapping** — checking for exposed OpenAPI/Swagger docs, GraphQL introspection, and versioned endpoints (`/v1/`, `/v2/`, internal-sounding paths) that often get less scrutiny than the public-facing ones.

I treat this phase as building a map of *attack surface*, not a list of vulnerabilities. The temptation here is to start firing scanners and calling every interesting response a finding — that's exactly the instinct that produced my "no impact" and "by design" rejections.

Phase 5: Hypothesis-Driven Testing

This is where actual hunting happens, and it's deliberately the fifth step, not the first.

- I pick a target based on the triage from Phase 3, then spend time **understanding what the feature is supposed to do** before looking for what it does wrong — authorization boundaries, trust assumptions between services, anywhere user input crosses a privilege boundary.
- I form a specific hypothesis ("this endpoint likely doesn't re-check ownership on update") before testing it, rather than poking randomly and hoping something breaks.
- Automated scanning (I use `nuclei` with curated templates) plays a supporting role here — triage and coverage, not discovery. It tells me where to look closer, not what to report.

[This is the most personal phase — describe your actual hunting style: are you logic-first, IDOR-hunter, auth-flow specialist, etc.]

Phase 6: Document As You Go

This phase used to not exist. I'd test for hours, find something promising, and *then* start writing the report from memory — which is exactly how I ended up with the "weak report" rejection from last time.

Now, documentation happens in parallel with testing, not after it:

- A running notes file per target, timestamped, with every request/response pair that looks even slightly interesting.
- Screen recording turned on by default during active testing — easier to trim a good PoC video down than to recreate one after the fact.
- A one-line "why this matters" note written the moment I suspect something is real, while the context is still fresh in my head.

By the time I decide something is reportable, 80% of the report already exists. The remaining 20% is just structuring it cleanly — which, not coincidentally, is exactly what triage teams are asking for.

What Six Phases Actually Buy You

None of this makes me find bugs faster. If anything, it slows down the first few hours on any new target. What it actually buys is fewer wasted hours on the wrong target, fewer reports that die on a technicality, and reports that read like someone who knew what they were doing — because by the time I write them, I do.

The rejections from my last post weren't a skill problem. They were a process problem. This is the process that replaced the guessing.

Next up: [how I actually find freelance pentest clients without a sales background] — subscribe so you don't miss it.

What does your recon process look like — more structured than this, or more instinct-driven? Curious how different people sequence this.